Math for Data Science - Lab 4

Course: Math for Data Science

This lab closes out the probability half of the course and opens the calculus half. We finish the distribution tour lab 3 began with the **Poisson** - including the sense in which it approximates the binomial - meet the **CDF** as the other way of describing a distribution, and use it to find the distribution of a *function* of a random variable. Then we turn to **derivatives**: what the limit definition actually computes, how the rules follow from it, and how to use the first and second derivative to find and classify the extrema of a function. Finally we pick up **L'Hôpital's rule**, the tool for the limits the derivative rules cannot reach - and use it to prove the $\lim_{n\to\infty}(1 + x/n)^n = e^x$ limit that sits underneath Exercise 1's binomial-to-Poisson picture.

From here on, calculus is a tool rather than a topic. Everything we do with continuous random variables in the next lab rests on it.

This is a **conceptual** lab, not a coding one. Every code cell comes with the scaffolding written for you and one or two lines left blank, so the work is deciding *what* goes in the gap - and then running the cell and playing with it. None of it asks you to write a program from scratch.

Exercises 1-5 are the core of the lab; exercise 6 - the St. Petersburg paradox - is optional, and looks back at expectation with the calculus in hand to ask what it means for a mean to be infinite. A few individual questions inside the core exercises are marked *[optional]* too, and can be skipped if you are short of time.

Setup

Run the cell below once to import everything we need.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Using R instead of Python (optional)

The lab is written in Python, but you can write your answers in R inside this same notebook. This needs R 4.5 or newer, and Jupyter started with rpy2 added: `uv run --with rpy2 jupyter lab`. The `README.md` of the course folder has the details and what to do when it does not work.

If you want to use R instead of Python, uncomment and run the following cell:

```
# %load_ext rpy2.ipython
```

Then, in any subsequent code cell, add `%%R` as the first line to run that cell as R:

```
%%R
x <- rbinom(1000, size = 10, prob = 0.3)
hist(x)
```

Keep running the given Python cells (imports, data loading) as they are. The hints name Python functions, such as `rng.binomial`; you will need to find the R equivalent (`rbinom`) yourself.

Exercise 1: The Poisson Distribution

The last distribution you need to meet for now is the **Poisson** distribution, used to model the number of successes that occur in a particular interval of time or space. For instance:

- The number of emails you will receive in an hour
- The number of buses that will go past you if you sit at the bus stop all day
- The number of major earthquakes that will happen in North America over the next decade
- The number of chips in a chocolate chip cookie

The PMF of the Poisson distribution is given by

$$P(X = k) = \frac{e^{-\lambda}\lambda^k}{k!}, \quad k = 0, 1, 2, \ldots$$

The only parameter you need is λ, the average rate of occurrence of the event: emails per hour, buses per day, earthquakes per year, or chips per cookie.

A cool fact about the Poisson is that it can be used to approximate the binomial where n is large and p is small. To map between the two distributions, set $\lambda = \sum_{j=1}^{n} p_j$, where j indexes intervals and p is the binomial probability parameter.

1. When you divide the city of Berlin into an 8×8 grid, the probability of a crime occurring in any sector of the city on any given day is $p = 0.1$. Generate a random vector of crime occurrence for a single day in Berlin by taking one Bernoulli draw per sector, then sum it to get the day's total.

```
rng = np.random.default_rng(123)

# One Bernoulli draw per sector: 64 sectors, a crime in each with probability 0.1.
# Fill in the draw. `rng.choice([0, 1], size=..., p=[prob_of_0, prob_of_1])`
does it
# in one line - mind the order of the two probabilities.
# here your code

print(bin_samp)
print("total crimes:", bin_samp.sum())
```

2. Now repeat the same thing by sampling from the Poisson.

With 64 sectors each at $p = 0.1$, the matching rate is $\lambda = 64 \times 0.1 = 6.4$.

```
# One Poisson draw at the matching rate. `rng.poisson(lam=...)` takes the rate
directly.
# here your code

print("total crimes (Poisson):", total)
```

3. Make a small simulation where you generate the total number of crimes on 10,000 random days in Berlin, once using binomial sampling and another time using Poisson sampling. Plot the histograms of the two distributions to compare them.

```
n_days = 10_000

# Two samples of `n_days` daily totals: one binomial, one Poisson. `rng.binomial`
# takes `n`, `p` and `size`; `rng.poisson` takes `lam` and `size`.
# here your code

# --- plotting, written for you: change it freely once it runs ---
bins = np.arange(-0.5, max(bin_samp.max(), pois_samp.max()) + 1.5)

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.hist(bin_samp, bins=bins, color="gray", alpha=0.5, label="Binomial(64, 0.1)")
ax.hist(pois_samp, bins=bins, color="blue", alpha=0.25, label="Poisson(6.4)")
ax.set_xlabel("Number of crimes")
ax.set_ylabel("Frequency")
ax.legend()
fig.tight_layout()
plt.show()
```

4. Now subdivide the city into more, smaller sectors by drawing a 50×50 grid. Smaller sectors mean a smaller chance of a crime in any one of them, so set $p = 0.00256$ - chosen so that the city-wide rate $\lambda = np$ stays at 6.4, exactly where it was on the 8×8 grid. The city is no more dangerous than before; we have only changed how finely we chop it up. Repeat the simulation and plot the histograms. What do you notice?

```
# The same two lines as question 3, with the finer grid: 2500 sectors at p
= 0.00256.
# The Poisson line does not change at all - that is the point.
# here your code

# --- plotting, written for you ---
bins = np.arange(-0.5, max(bin_samp.max(), pois_samp.max()) + 1.5)
```

```
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.hist(bin_samp, bins=bins, color="gray", alpha=0.5, label="Binomial(2500,
0.00256)")
ax.hist(pois_samp, bins=bins, color="blue", alpha=0.25, label="Poisson(6.4)")
ax.set_xlabel("Number of crimes")
ax.set_ylabel("Frequency")
ax.legend()
fig.tight_layout()
plt.show()
```

here your solution

5. The Poisson model of the crime counts rests on two assumptions: that the sectors are **independent** of one another, and that the rate is **constant** across them. Crime in a real city is not spread evenly - it clusters, in a few sectors and a few hours. Which of the two assumptions does that break, and which way would you expect the variance of the daily total to move relative to the Poisson's λ?

here your solution

Exercise 2: The Cumulative Distribution Function

So far we have described discrete distributions with the PMF, $P(X = x)$. There is a second description that works for *every* random variable, discrete or continuous, and it is the one that will carry us into the next lab: the **cumulative distribution function**

$$F_X(x) = P(X \leq x)$$

1. Let $X \sim \text{Binomial}(4, 1/2)$. Compute its PMF by hand for $x = 0, \ldots, 4$, then build the CDF from it.

here your solution

2. Reproduce the table in code with `np.cumsum`, and plot the PMF and the CDF side by side. Draw the CDF as a **step** function - that is what it is.

> **Tip:** `math.comb(n, k)` gives the binomial coefficient; `ax.step(..., where="post")` draws a right-continuous step.

```
from math import comb

k = np.arange(5)

# The PMF is C(4, i) / 16 for each i in k, and the CDF is its running total.
```

```
# `np.cumsum` does the running total in one call.
# here your code

print(pd.DataFrame({"k": k, "P(X=k)": pmf, "F(k)": cdf}))

# --- plotting, written for you ---

fig, axes = plt.subplots(1, 2, figsize=(9, 3.8))
axes[0].stem(k, pmf)
axes[0].set_title("PMF  $P(X = k)$")
axes[0].set_ylim(0, 1)

# extend the grid on both sides so the flat tails of the CDF are visible
grid = np.arange(-1, 6)
cdf_grid = np.concatenate([[0], cdf, [1]])
axes[1].step(grid, cdf_grid, where="post", color="red")
axes[1].set_title("CDF  $F(x) = P(X \\leq x)$")
axes[1].set_ylim(0, 1.05)

for ax in axes:
    ax.set_xlabel("$x$")
fig.tight_layout()
plt.show()
```

3. The jump in the CDF at $x = k$ has height $P(X = k)$ - the PMF and the CDF carry exactly the same information. Verify this in code by recovering the PMF from the CDF with `np.diff`.

Then use the CDF to compute $P(1 < X \leq 3)$, using $P(x_1 < X \leq x_2) = F_X(x_2) - F_X(x_1)$.

```
# The jumps of the CDF are the PMF. `np.diff` gives the gaps between consecutive
# entries; prepending a 0 with `np.concatenate([[0], cdf])` is what makes the
first
# jump come out right.
# here your code

print("recovered PMF:", np.round(recovered, 4))
print("original PMF: ", np.round(pmf, 4))
print("match?", np.allclose(recovered, pmf))

# P(1 < X <= 3) = F(3) - F(1). Index `cdf` to pick out the two values you need.
# here your code
```

```
print(f"\nP(1 < X <= 3) = {interval_prob:.4f}")
print(f"direct check, P(X=2) + P(X=3) = {pmf[2] + pmf[3]:.4f}")
```

4. Now read two things straight off the step plot you drew in question 2, *without* going back to the PMF: $P(X > 2)$ and $P(X = 2)$. Say which feature of the picture each one is - a height, a jump, a gap to the top.

Then argue from the definition $F_X(x) = P(X \leq x)$ why F must be **non-decreasing** and **right-continuous**, and what would have to be true of a "CDF" that dropped somewhere.

here your solution

5. The CDF also gives you a clean route to the distribution of a *function* of a random variable. Roll two fair six-sided dice and let $M = \max(X, Y)$. There are 36 equally likely outcomes.

Find $F_M(k) = P(M \leq k)$ first - ask yourself what has to be true of *both* dice for the maximum to be at most k - and then difference it, exactly as you just did with `np.diff`, to show that

$$P(M = k) = \frac{2k - 1}{36}, \qquad k = 1, \ldots, 6$$

here your solution

6. Verify the PMF by simulating many pairs of rolls, and plot the simulated PMF against the exact one.

```
rng = np.random.default_rng(7)

n_rolls = 200_000
d1 = rng.integers(1, 7, size=n_rolls)
d2 = rng.integers(1, 7, size=n_rolls)
m_vals = np.arange(1, 7)

# Two lines: the elementwise maximum of the two dice (`np.maximum`), and the exact
# PMF from the formula above, evaluated on the whole array `m_vals` at once.
# here your code

simulated = np.array([np.mean(m == i) for i in m_vals])

for i, s, e in zip(m_vals, simulated, exact):
    print(f"P(M = {i}):  simulated {s:.4f}   exact {e:.4f}")

# --- plotting, written for you ---
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.bar(m_vals, simulated, color="steelblue", alpha=0.6, label="simulated")
ax.plot(m_vals, exact, "o-", color="red", label="exact $(2k-1)/36$")
```

```
ax.set_xlabel("$k$")
ax.set_ylabel("$P(M = k)$")
ax.legend()
fig.tight_layout()
plt.show()
```

7. Why does the CDF matter, if it carries the same information as the PMF? Think about what happens when X can take *any* real value in an interval.

here your solution

Exercise 3: Derivatives - What the Limit Definition Computes

The derivative of f at a is defined as a limit of slopes of secant lines:

$$f'(a) = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h}$$

Everything else in differential calculus - the power rule, the chain rule, the whole lot - is a consequence of this one definition. Let's watch it converge.

1. Take $f(x) = x^2 + 1$ and $a = 1$. Compute the difference quotient $\frac{f(1+h)-f(1)}{h}$ for $h = 1, 0.1, 0.01, 0.001, 10^{-6}$. What number are the secant slopes heading towards?

```
def f(x):
    return x ** 2 + 1


a = 1.0
for h in (1, 0.1, 0.01, 0.001, 1e-6):
     # The difference quotient itself - write the formula above as one line
of Python.
    # here your code
    print(f"h = {h:<8} secant slope = {slope:.8f}")
```

here your solution

2. Draw the picture. Plot f over $[-1, 3]$ together with the secant lines through $(1, f(1))$ and $(1 + h, f(1 + h))$ for a few shrinking values of h, and the tangent line $y = 2(x - 1) + 2$ on top.

```
grid = np.linspace(-1, 3, 400)

fig, ax = plt.subplots(figsize=(7.5, 5))
ax.plot(grid, f(grid), color="black", linewidth=2, label="$f(x) = x^2 + 1$")

for h, colour in zip((1.5, 1.0, 0.5, 0.25), ("#cccccc", "#999999", "#666666",
```

```
"#333333")):
   # Two lines: the secant's slope (the difference quotient again), and the secant
    # line itself evaluated on `grid`, in point-slope form y = slope * (x - a)
+ f(a).
    # here your code
    ax.plot(grid, secant, color=colour, linewidth=1, label=f"secant, $h = {h}$")
    ax.scatter([a + h], [f(a + h)], color=colour, zorder=3)

# the tangent, given: slope 2, through (a, f(a))
ax.plot(grid, 2 * (grid - a) + f(a), color="red", linewidth=2, label="tangent,
slope 2")
ax.scatter([a], [f(a)], color="red", zorder=4)
ax.set_xlabel("$x$")
ax.set_ylabel("$y$")
ax.set_ylim(-1, 10)
ax.legend(fontsize=8)
fig.tight_layout()
plt.show()
```

3. Differentiate the following by hand, naming the rule you use at each step.

a. $f(x) = 3x^4 - 2x + 7$

b. $g(x) = (x^2 + 1)^5$

c. $h(x) = x^2 e^x$

d. $k(x) = \ln(3x^2 + 1)$

e. $m(x) = e^{-x^2/2}$ - remember this one; it is the shape of the Normal density.

here your solution

4. Check your five answers numerically. Write a function `numerical_derivative(f, x, h)` that returns the **central** difference quotient

$$\frac{f(x+h) - f(x-h)}{2h}$$

and compare it against your hand-derived formulas at a few points.

> **Tip:** the central difference is more accurate than the one-sided version we started with - its error shrinks like h^2 rather than h. Use $h = 10^{-5}$ or so; going much smaller makes things *worse*, because subtracting two nearly equal floating-point numbers loses precision.

```
def numerical_derivative(fn, x, h=1e-5):
     """The central difference quotient - one line, straight from the formula
above."""
    # here your code


test_points = np.array([-1.0, 0.5, 2.0])

# the five functions from question 3, written for you
functions = [
    ("3x^4 - 2x + 7", lambda x: 3 * x ** 4 - 2 * x + 7),
    ("(x^2+1)^5",     lambda x: (x ** 2 + 1) ** 5),
    ("x^2 e^x",       lambda x: x ** 2 * np.exp(x)),
    ("ln(3x^2+1)",    lambda x: np.log(3 * x ** 2 + 1)),
    ("exp(-x^2/2)",   lambda x: np.exp(-x ** 2 / 2)),
]

# Your five hand-derived derivatives from question 3, in the same order. This
is the
# only real work in the cell - `np.exp` and `np.log` are the ones you need.
# here your code

for (name, fn), exact in zip(functions, derivatives):
    num = numerical_derivative(fn, test_points)
    ana = exact(test_points)
      print(f"{name:<14} max |numerical - analytic| = {np.max(np.abs(num -
ana)):.2e}")
```

5. *[optional]* The tip above warns that shrinking h past about 10^{-5} makes the answer *worse*, not better - a difference quotient at $h = 10^{-12}$ is less accurate than at $h = 10^{-5}$, even though the limit is taken as $h \to 0$. Two different error sources are at work here, pulling in opposite directions. Name them, say which one dominates at large h and which at small h, and explain why that means there is a best h somewhere in between.

here your solution

Exercise 4: Derivatives and the Shape of a Graph

The first derivative tells you where a function is going up or down; the second tells you how it bends. Together they locate and classify the extrema - which is the entire engine behind maximum likelihood estimation later in the course.

Recall the procedure:

1. Find the **critical points**: the x where $f'(x) = 0$ (or f' does not exist).

2. **First derivative test**: if f' changes $+ \to -$ at a critical point, it is a local maximum; $- \to +$ makes it a local minimum; no sign change makes it neither.
3. **Second derivative test**: at a critical point c, $f''(c) < 0$ means a local maximum, $f''(c) > 0$ a local minimum, and $f''(c) = 0$ is inconclusive.

1. Let $f(x) = x^3 - 3x^2 + 1$. Find f' and f'', locate the critical points, and classify them with both tests. Where is f concave up, and where concave down?

here your solution

2. Plot f over $[-1.5, 3.5]$. Mark the two critical points and the inflection point, and colour the curve by the sign of f' so the increasing and decreasing stretches are visible.

```
def f3(x):
    return x ** 3 - 3 * x ** 2 + 1


def df3(x):
    """f' from question 1 - one line."""
    # here your code


# The critical points and the inflection point, straight off your answer to
question 1.
# here your code

grid = np.linspace(-1.5, 3.5, 500)
y = f3(grid)

# --- plotting, written for you: colours the curve by the sign of f' ---
fig, ax = plt.subplots(figsize=(8, 5))

rising = df3(grid) > 0
ax.plot(np.where(rising, grid, np.nan), np.where(rising, y, np.nan),
        color="green", linewidth=2.5, label="$f' > 0$ (increasing)")
ax.plot(np.where(~rising, grid, np.nan), np.where(~rising, y, np.nan),
        color="red", linewidth=2.5, label="$f' < 0$ (decreasing)")

ax.scatter(critical_x, [f3(c) for c in critical_x], color="black", zorder=5,
s=60,
           label="critical points")
ax.scatter(inflection_x, [f3(c) for c in inflection_x], color="blue", zorder=5,
s=60,
           marker="s", label="inflection point")
ax.axhline(0, color="gray", linewidth=0.5)
```

```
ax.set_xlabel("$x$")
ax.set_ylabel("$f(x)$")
ax.legend(fontsize=9)
fig.tight_layout()
plt.show()
```

3. Confirm the classification numerically, without doing any algebra: evaluate f on a fine grid, find where the numerical derivative changes sign, and check the sign of the numerical second derivative there.

> **Tip:** `np.gradient(y, x)` differentiates a sampled function. Applying it twice gives you an approximate f''.

```
# The numerical first and second derivative. `np.gradient(values, positions)`
# differentiates once; apply it to its own output to differentiate again.
# here your code

# where does f' change sign between consecutive grid points? (written for you)
sign_change = np.where(np.diff(np.sign(d1)) != 0)[0]

for i in sign_change:
    x_c = grid[i]
      # The second derivative test, in one line: which sign of d2[i] means
which kind?
    # here your code
      print(f"critical point near x = {x_c:+.3f}    f'' = {d2[i]:+.3f}    ->
local {kind}")
```

4. The second derivative test says that $f''(c) = 0$ at a critical point is **inconclusive**. That is a stronger statement than "we do not know how to decide": it says the value $f''(c) = 0$ genuinely carries no information about what happens at c. Show this by finding three functions, each with a critical point at $c = 0$ where $f''(0) = 0$, for which that point is

a. a local minimum,

b. a local maximum,

c. neither.

Which test does settle each case?

here your solution

5. *[optional]* One more, and this one is a preview. The likelihood of observing k successes in n independent trials, as a function of the unknown success probability p, is

$$L(p) = p^k(1-p)^{n-k}, \qquad 0 < p < 1$$

Take $n = 10$, $k = 3$. Plot L, find the p that maximizes it by setting $L'(p) = 0$, and check the answer against the plot. Does the result surprise you?

> **Hint:** differentiating $\ln L$ instead of L is much easier and gives the same maximizer, because $\ln$ is strictly increasing. That trick is the whole reason the "log-likelihood" exists.

```
n, k_obs = 10, 3
p_grid = np.linspace(0.001, 0.999, 500)

# The likelihood L(p) = p^k (1-p)^(n-k), evaluated on the whole grid at once.
# here your code

# --- plotting, written for you ---
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(p_grid, L, color="blue")
ax.axvline(k_obs / n, color="red", linestyle="--", label=f"$p = k/n = {k_obs /
n}$")
ax.set_xlabel("$p$")
ax.set_ylabel("$L(p)$")
ax.legend()
fig.tight_layout()
plt.show()

print(f"argmax on the grid: {p_grid[np.argmax(L)]:.3f}")
```

here your solution

Exercise 5: L'Hôpital's Rule

Some limits are out of reach of the rules we have. $\lim_{x\to 0} \frac{e^x - 1}{x}$ is $\frac{0}{0}$ on substitution, and the quotient rule is no help - it *differentiates* a quotient, it does not *evaluate* one. **L'Hôpital's rule** is the tool for exactly this case:

> If f and g are differentiable near a with $g'(x) \neq 0$ there, and $\lim_{x\to a} f(x) = \lim_{x\to a} g(x)$ is either 0 or $\pm\infty$, then
>
> $$\lim_{x\to a} \frac{f(x)}{g(x)} = \lim_{x\to a} \frac{f'(x)}{g'(x)}$$
>
> whenever the limit on the right exists. It holds for $x \to \pm\infty$ and for one-sided limits too.

The rule is a race between two slopes: near a each function is close to its tangent line, and the $(x-a)$ that made *both* vanish cancels. That is why differentiating top and bottom separately - nonsense anywhere else - is the right move here.

This exercise is done by hand. Only the last question asks for code.

1. Evaluate the following. Before **every** application of the rule, say what indeterminate form you are looking at - a limit can stop being indeterminate after a single step, and applying the rule to something that is no longer $\frac{0}{0}$ or $\frac{\infty}{\infty}$ gives the wrong answer (question 2 is that trap).

a. $\lim_{x\to 0} \frac{1-\cos x}{x^2}$

b. $\lim_{x\to \infty} \frac{\ln x}{x}$

c. $\lim_{x\to \infty} \frac{x^3}{e^x}$

d. $\lim_{x\to 0^+} x \ln x$ - not a quotient yet. Rewrite the $0 \cdot \infty$ form by dividing by a reciprocal; only one of the two choices leads anywhere.

here your solution

2. The trap. Consider

$$\lim_{x\to 0} \frac{x+1}{2x+1}$$

Differentiating top and bottom gives $\lim_{x\to 0} \frac{1}{2} = \frac{1}{2}$. But the true limit is 1. In a sentence or two: what went wrong, and what is the rule you should follow to avoid it?

here your solution

3. The payoff. Show that

$$\lim_{n\to\infty} \left(1 + \frac{x}{n}\right)^n = e^x$$

This is of the form 1^∞ - the base tends to 1 while the exponent runs off to infinity - which is indeterminate, and again not a quotient. The route is: take logs, turn the result into the quotient $\frac{\ln(1+x/n)}{1/n}$, apply the rule, and exponentiate back at the end.

> Treat n as a continuous variable for the purpose of differentiating; the sequence limit follows from the function limit.

here your solution

4. *[optional]* Tie it back to Exercise 1. You saw there, by simulation, that a $\text{Binomial}(n, p)$ with n large and p small looks like a $\text{Poisson}(\lambda = np)$. Question 3 is what makes that true analytically. Fix $\lambda > 0$, set $p = \lambda/n$, and show that for each fixed k

$$\binom{n}{k}\left(\frac{\lambda}{n}\right)^k\left(1-\frac{\lambda}{n}\right)^{n-k} \to \frac{e^{-\lambda}\lambda^k}{k!} \qquad \text{as} \quad n\to\infty$$

Hint: split the expression into three pieces - the falling factorial $n(n-1)\cdots(n-k+1)/n^k$, the factor $(1-\lambda/n)^n$, and the factor $(1-\lambda/n)^{-k}$ - and take the limit of each separately.

here your solution

5. *[optional]* Check question 3 numerically. Evaluate $(1+x/n)^n$ for n growing over several orders of magnitude, at two or three values of x, and plot the absolute error against e^x with a log scale on both axes. What is the slope of the error curve telling you - and does it keep falling all the way?

Tip: `np.logspace(0, 15, 60)` gives you the grid of n. Remember Exercise 3's question 5: this is a difference of nearly equal floating-point numbers too, so do not be surprised if the error stops improving at the far end.

```
n_grid = np.logspace(0, 15, 60)

fig, ax = plt.subplots(figsize=(7.5, 4.8))

for x_val, colour in zip((1.0, 2.0, -1.0), ("blue", "green", "red")):
    # Two lines: the approximation (1 + x/n)^n on the whole grid, and its absolute
    # distance from the true value e^x. `np.abs` and `np.exp` are what you need.
    # here your code
    ax.plot(n_grid, error, color=colour, label=f"$x = {x_val}$")

# --- the rest is written for you: log axes are what make the slope readable ---
ax.set_xscale("log")
ax.set_yscale("log")
ax.set_xlabel("$n$")
ax.set_ylabel(r"$|(1 + x/n)^n - e^x|$")
ax.legend()
fig.tight_layout()
plt.show()

# the value at a few n, for x = 1
for n_val in (10, 100, 10_000, 10**8, 10**15):
    print(f"n = {n_val:>17,}   (1 + 1/n)^n = {(1 + 1 / n_val) ** n_val:.12f}")
print(f"{'e':>21}  = {np.e:.12f}")
```

here your solution

Exercise 6: Going Further - The St. Petersburg Paradox

[optional]

One last look back at expectation, now that you have the machinery of limits in hand. Everything that goes wrong below is a limit that fails to exist: an infinite expectation is a divergent series, and the running average you are about to plot is a sequence with nowhere to converge to.

A casino offers you the following game. It flips a fair coin until the first heads appears. If the first heads comes up on flip k, you are paid 2^k euros. How much would you pay to play?

The expected payout is

$$E(X) = \sum_{k=1}^{\infty} 2^k \cdot \left(\frac{1}{2}\right)^k = 1 + 1 + 1 + \ldots = \infty$$

so on expectation grounds you should be willing to pay *any* finite entry fee. Almost nobody would. Let's see what the simulation says.

1. Simulate 100,000 plays of the game and plot the running average payout against the number of plays. Compare this with the running average of something ordinary - say a die roll.

> **Tip:** `np.cumsum(payouts) / np.arange(1, n+1)` gives you the running average. A log scale on the y-axis helps.

```
rng = np.random.default_rng(1848)

n_plays = 100_000
plays = np.arange(1, n_plays + 1)

# the number of flips until the first heads is Geometric (written for you)
k = rng.geometric(p=0.5, size=n_plays)

# The payout for a first heads on flip k, and the running average of the payouts.
# `np.cumsum(values) / plays` is the running-average idiom from the tip.
# here your code

# the same running average for an ordinary die roll, to compare against
running_die = np.cumsum(rng.integers(1, 7, size=n_plays)) / plays

# --- plotting, written for you ---
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(plays, running_stp, color="blue", label="St. Petersburg payout")
ax.plot(plays, running_die, color="gray", label="die roll (for comparison)")
ax.axhline(3.5, color="red", linestyle="--", linewidth=1, label="$E$(die roll)
$= 3.5$")
ax.set_xscale("log")
ax.set_yscale("log")
```

```
ax.set_xlabel("Number of plays")
ax.set_ylabel("Running average payout")
ax.legend()
fig.tight_layout()
plt.show()
```

2. What do you see, and what does it say about the average payout as a summary of this game?

here your solution